게임 프로그래머 포트폴리오



박대원

Tel. : 010 – 9961 - 7708

E-Mail : june25656@gmail.com

깃허브 주소

https://github.com/DreamDaeWon?tab=stars

유튜브 주소

PROJECTS

SlayTheLord (S.T.L) 게임공학과 졸업작품

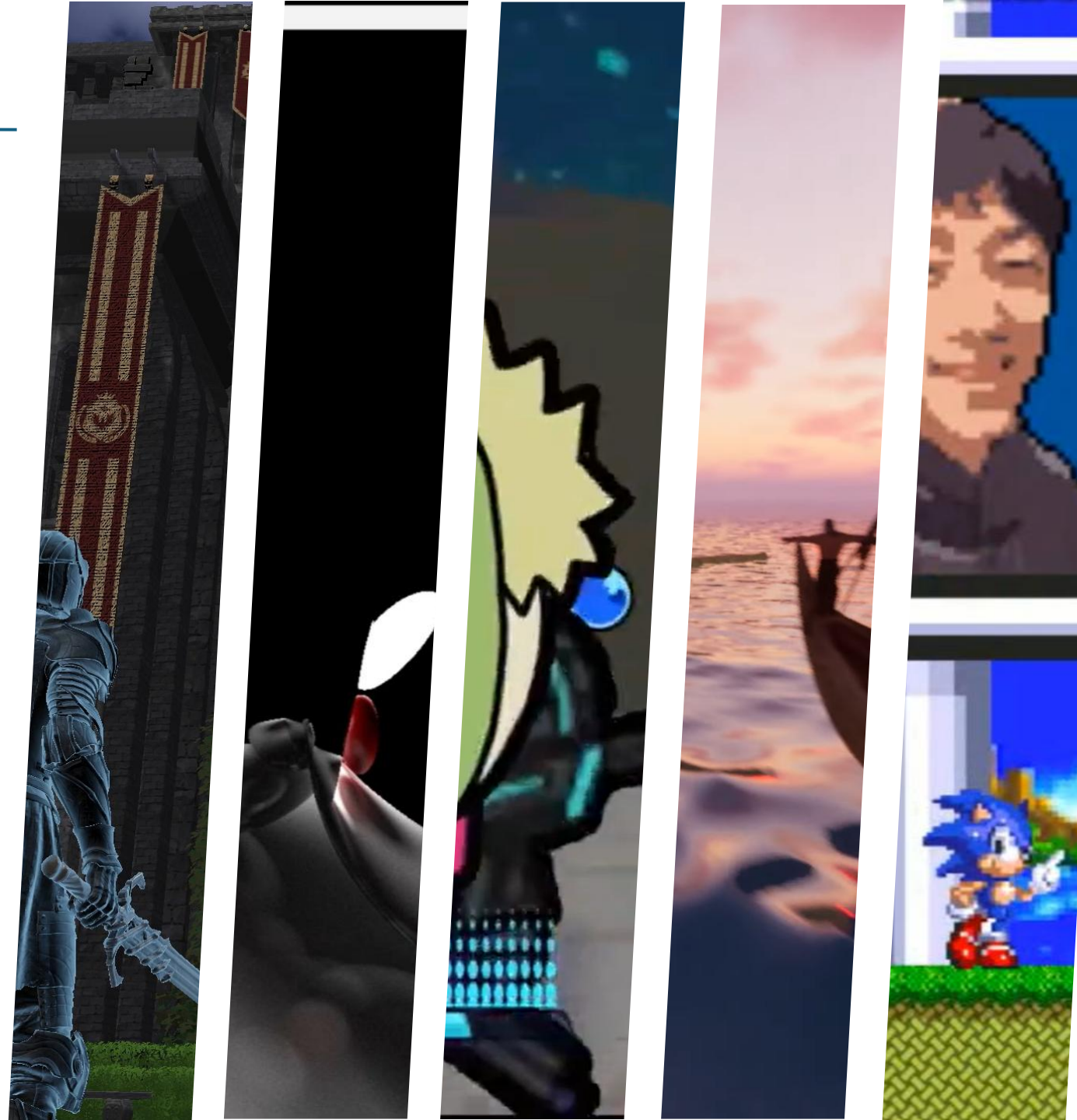
- 프로젝트 개요
- 영상
- DirectX 12 Core Architecture : Linear Allocator
- Custom Skeletal Animation Mesh glTF Parser
- Animation Component & Master Skeleton
- Socket Component & Attachment System
- Compute Shader Particle System
- 한국게임학회 춘계 학술발표대회 논문

RayTracingSimulator 컴퓨터그래픽스 텀프로젝트

- 프로젝트 개요
- 영상
- 실시간 렌더링 노이즈 최적화
- 실시간 패스 트레이싱 고도화 및 신경망 광선 재구성 (DLSS RR)
- DLSS RR 적용 전/후 비교화면

기타 프로젝트

- Riffle Effect 모작 (2.5D, DirectX 9)
- 2D MMORPG IOCP Server (2D, WinAPI, IOCP)
- Moana Tower Defense (3D, 언리얼)
- Sonic the Hedgehog 모작 (2D, WinAPI)
- CupHeadBanging (2D, Python)
- Geometry Dash 모작 (2D, WinAPI)
- TheGreatestNature (3D, 언리얼)



2026 한국공학대학교 게임공학과 졸업작품 우수작 선정

SlayTheLord







SlayTheLord

프로젝트 개요













프로젝트 이름	S.T.L (Slay The Lord)
장르	4인 협동 멀티 플레이어 3인칭 다크 판타지 Action RPG
설명	파티와 함께 퀘스트를 수행하며, 협동 플레이를 통해 빼앗긴 마을을 되찾는 여정을 다룬 다크 판타지 게임
개발 인원	3명 (서버 프로그래밍1, 클라이언트 프로그래밍2)
담당 역할	클라이언트 엔진 메모리 최적화, glTF 직접 파싱, 스키닝 애니메이션, 컴퓨트 셰이더 기반 파티클 시스템, 카메라 충돌처리, 인게임 컷씬 연출 제작
개발 언어	C++20, HLSL
사용 툴 & 라이브러리	Visual Studio 2022, VS Code, DirectX 12, Jolt Physics, ImGui (ImGuizmo), JSON, Git, GitHub Desktop, FMOD
활용 AI	Claude, Gemini, Antigravity (알고리즘 검증 및 디버깅 보조)
제작 기간	2025 / 08 ~ 2026 / 06

SlayTheLord

인게임 플레이 영상



https://youtu.be/x3_Z6sM7qkM?si=ofY-ANeH2R0508p7

DirectX 12 Core Architecture : Linear Allocator

[? 문제점] 애니메이션 컴포넌트에서 뼈대 행렬 관리 시 매 프레임 수많은 상수 버퍼(Bone) Map/Unmap 시 심각한 CPU 병목 발생

[→ 해결책] Persistent Mapping & Frame Allocator

- 32MB 단일 버퍼 생성 후 Persistent Mapping (Map 호출 1회로 고정)
- 더블 버퍼링 기반 프레임별 메모리 파티셔닝 (16MB 단위 분할)
- 256-Byte 정렬을 통한 O(1) 메모리 할당 및 고속 memcpy (오버헤드 0)




```cpp
LinearAllocator::Allocation LinearAllocator::allocate(size_t size)
{
    // 할당할 크기를 256바이트 배수로 올림 연산
    size_t alignedSize = (size + CB_ALIGNMENT) & ~CB_ALIGNMENT;

    // 현재 프레임에 할당된 메모리 범위를 초과하는지 검사 (메모리 부족 방어)
    size_t frameMaxBoundary = (_currentFrameIndex + 1) * _frameSize;
    if (_currentOffset + alignedSize > frameMaxBoundary)
    {
        // 이 로그가 뜬다면 초기 설정한 32MB 용량이 부족하다는 뜻
        CERROR("LinearAllocator Out of Memory in current frame!");
        return { nullptr, 0 };
    }

    // 할당할 위치 계산
    Allocation alloc;
    alloc.cpuPtr = static_cast<uint8_t*>(_cpuBase) + _currentOffset;
    alloc.gpuAddr = _gpuBase + _currentOffset;

    // 오프셋 증가 (다음 NPC가 쓸 주소 갱신)
    _currentOffset += alignedSize;

    return alloc;
}
```

O(1) 선형 할당(Linear Allocate) 핵심 로직

→ 상수 버퍼의 256-Byte 정렬 및 단일 포인터(Offset) 전진 연산

```cpp
size_t joint_size = gltf_mesh->get_joint_count();
UINT element_size = sizeof(DirectX::XMFLOAT4X4);

// DW설명 : 단순히 뼈대 행렬의 개수 * 행렬 크기로 버퍼 크기를 계산함
_bonePaletteSize = joint_size * element_size;
```

실제 애니메이션 컴포넌트에서는 할당받을 크기만 계산

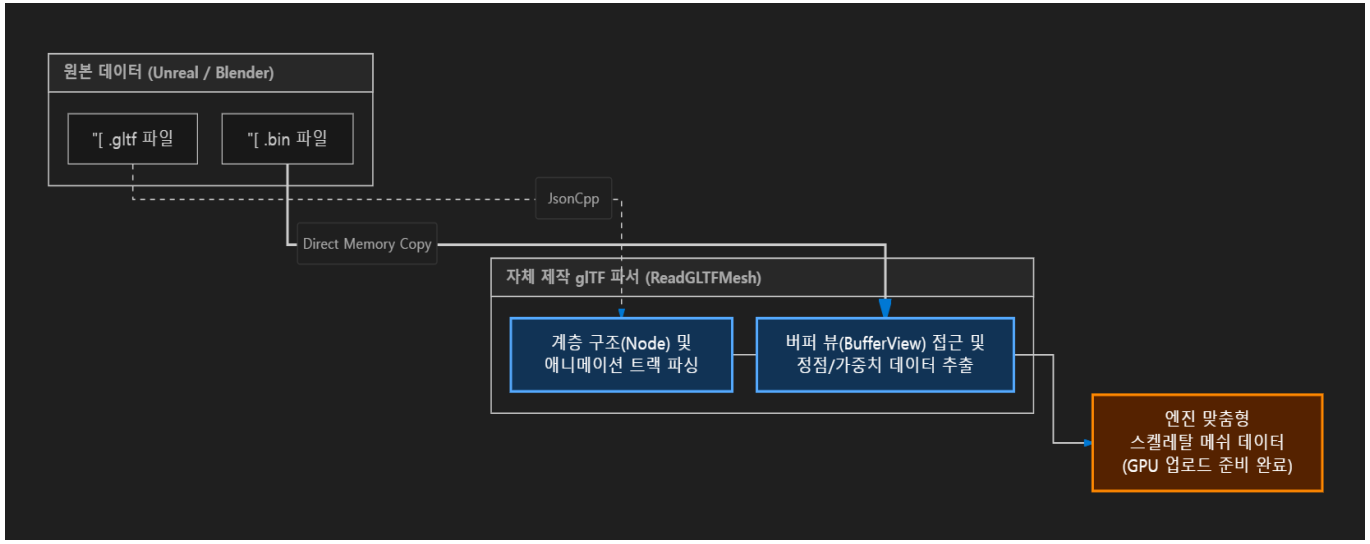


Custom Skeletal Animation Mesh glTF Parser

[? 문제점] Assimp 등 외부 라이브러리 사용 시 엔진에서 사용하지 않는 메타데이터(조명, 카메라 등) 및 무거운 범용 트리(aiNode) 객체 생성으로 인한 이중 할당 오버헤드 발생 및 로딩 속도 저하

[→ 해결책] glTF 구조를 분석하여 자체 glTF 바이너리 파서(Binary Parser) 구현

- 중간 계층 생성 생략: 범용 라이브러리의 무거운 트리 구조 ex)aiScene 파싱을 생략하고, 바이너리 버퍼 다이렉트 매핑을 통해 로딩 시간 극한 단축
- 선택적 파싱: 엔진 구동에 필수적인 데이터(정점, 인덱스, 뼈대 가중치, 계층 구조)만 추출하여 메모리 파편화 방지
- 상용 엔진(Unreal, Unity)의 모델 데이터를 우리 게임 엔진 규격에 맞게 100% 최적화하여 로드




```cpp
// JOINTS_0 읽기 (u8, u16 대응 + Stride 적용)
if (primitive_json["attributes"].contains("JOINTS_0"))
{
    int accessor_idx = primitive_json["attributes"]["JOINTS_0"];
    const json& accessor = gltf_json["accessors"][accessor_idx];
    const json& buffer_view = gltf_json["bufferViews"][accessor["bufferView"].get<int>()];

    const char* buffer_start =
        binary_buffer.data()
        + buffer_view.value("byteOffset", 0)
        + accessor.value("byteOffset", 0);

    int count = accessor["count"];
    int component_type = accessor["componentType"]; // 5121(u8), 5123(u16)

    // 버퍼 뷰의 Stride를 가져오기 (없으면 0)
    int byte_stride = buffer_view.value("byteStride", 0);

    joint_indices_vec.resize(count);

    for (int i = 0; i < count; ++i)
    {
        const char* current_ptr = nullptr;

        if (component_type == 5123) // Unsigned Short (2 byte)
        {
            // Stride가 0이면 요소 크기(2*4=8) 사용, 아니면 Stride 사용
            int step = (byte_stride > 0) ? byte_stride : (sizeof(uint16_t) * 4);

            // i번째 요소의 정확한 메모리 위치 계산
            current_ptr = buffer_start + i * step;

            const uint16_t* ptr = reinterpret_cast<const uint16_t*>(current_ptr);
            joint_indices_vec[i] = XMUINT4(ptr[0], ptr[1], ptr[2], ptr[3]);
        }
        else if (component_type == 5121) // Unsigned Byte (1 byte)
```

다이렉트 바이너리 매핑: Stride와 Offset 포인터 연산을 통한 데이터 고속 파싱



Animation Component & Master Skeleton

[? 문제점] 캐릭터, 갑옷, 투구 등 여러 파츠가 동일한 애니메이션을 재생할 때, 파츠별로 각각의 뼈대 행렬을 계산하면 CPU 연산 및 메모리 오버헤드 발생

[→ 해결책] 마스터 스켈레톤(Master Skeleton) 기반의 뼈대 인덱스 리매핑 및 버퍼 공유 시스템 구현

- Bone Index Remapping: 로컬 메쉬의 관절(Joint) 인덱스를 local_to_master_map을 통해 마스터 스켈레톤 인덱스로 변환 매핑
- O(1) 단일 팔레트 참조: 모든 장착 파츠들이 마스터 스켈레톤의 단일 Matrix Palette 상수 버퍼(Constant Buffer) 하나만 참조하도록 설계
- CPU 병목 제거: 캐릭터 파츠가 수십 개로 늘어나도, 애니메이션 행렬 보간 연산은 매 프레임 '단 1회'만 수행하여 성능 극대화

```
"skins": [
    {
        "name": "SK_Body",
        "inverseBindMatrices": 0,
        "skeleton": 0,
        "joints": [ 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20, 21, 22, 23, 24, 25, 26, 27, 28, 29, 30, 31, 32, 33, 34, 35, 36, 37, 38, 39, 40, 41, 42, 43, 44, 45
    },
    {
        "name": "SK_ArmBandages",
        "inverseBindMatrices": 1,
        "skeleton": 51,
        "joints": [ 52, 53, 54, 55, 56, 57, 58, 59, 60, 61, 62, 63, 64, 65, 66, 67, 68, 69, 70, 71, 72, 73, 74, 75, 76, 77, 78, 79, 80, 81, 82, 83, 84, 85, 86, 87, 88, 89, 90, 91, 92, 93,
    },
    {
        "name": "SK_Chest",
        "inverseBindMatrices": 2,
        "skeleton": 102,
        "joints": [ 103, 104, 105, 106, 107, 108, 109, 110, 111, 112, 113, 114, 115, 116, 117, 118, 119, 120, 121, 122, 123, 124, 125, 126, 127, 128, 129, 130, 131, 132, 133, 134, 135, 136,
    },
    {
        "name": "SK_Helmet",
        "inverseBindMatrices": 3,
        "skeleton": 153,
        "joints": [ 154, 155, 156, 157, 158, 159, 160, 161, 162, 163, 164, 165, 166, 167, 168, 169, 170, 171, 172, 173, 174, 175, 176, 177, 178, 179, 180, 181, 182, 183, 184, 185, 186, 187,
    },
    {
        "name": "SK_Legs",
        "inverseBindMatrices": 4,
        "skeleton": 204,
        "joints": [ 205, 206, 207, 208, 209, 210, 211, 212, 213, 214, 215, 216, 217, 218, 219, 220, 221, 222, 223, 224, 225, 226, 227, 228, 229, 230, 231, 232, 233, 234, 235, 236, 237, 238,
    },
    {
        "name": "SK_Shoulder_L",
        "inverseBindMatrices": 5,
        "skeleton": 255,
        "joints": [ 256, 257, 258, 259, 260, 261, 262, 263, 264, 265, 266, 267, 268, 269, 270, 271, 272, 273, 274, 275, 276, 277, 278, 279, 280, 281, 282, 283, 284, 285, 286, 287, 288, 289,
    },
```

←팔, 다리 갑옷, 투구 등의 파츠마다 각각 따로 뼈대의 정보가 존재

```
// 마스터 스켈레톤 매핑 테이블 생성
std::vector<int> local_to_master_map;
if (gltf_json.contains("skins") && skin_index < gltf_json["skins"].size())
{
    const json& current_skin = gltf_json["skins"][skin_index];
    const auto& local_joints = current_skin["joints"];

    local_to_master_map.resize(local_joints.size());

    for (size_t j = 0; j < local_joints.size(); ++j)
    {
        int local_node_idx = local_joints[j].get<int>();
        std::string bone_name = gltf_json["nodes"][local_node_idx].value("name", "unnamed");

        // 파츠의 j번째 뼈가 마스터 스켈레톤(몸통)의 몇 번째 뼈인지 찾아 기록
        local_to_master_map[j] = get_palette_index_by_name(bone_name);
    }
}
```

독립된 뼈대를 가진 서브 메쉬들을 Master Skeleton에 동기화 하는 매핑 테이블 생성



(자체 제작 툴 화면)팔, 다리 갑옷, 투구 등의 파츠마다 있는 스켈레톤이 마스터 스켈레톤과 동기화된 모습



Socket Component & Attachment System

[? 문제점] 캐릭터의 실시간 애니메이션(달리기, 공격 등) 프레임 변화에 맞춰 무기가 손의 궤적을 정확히 따라가도록 동기화해야 함

[→ 해결책] 행렬 팔레트(Matrix Palette) 추적 기반의 자체 Socket Component 시스템 구현

- 뼈대 행렬 추출: Animation Component에서 매 프레임 보간되어 계산된 특정 뼈대 (예: LeftHand)의 행렬 데이터 실시간 참조
- 실시간 행렬 곱셈 연산: 무기 Local 행렬 × 뼈대 Socket 행렬 × 캐릭터 World 행렬 연산 파이프라인을 통해 무기의 최종 월드 좌표 도출
- 커스텀 Attachment 시스템: 자체 엔진 환경에서 다중 객체 결합(Weapon, Armor 등)을 완벽하게 처리

```cpp
// 모든 연결된 객체들에 대해 위치 갱신
for (auto& pair : _connectedObjects)
{
    auto& socket_info = pair.second;
    int bone_index = mesh->get_bone_index_by_name(socket_info.bone_name);
    if (bone_index < 0) continue;

    // 소켓 오브젝트의 TransformComponent 가져오기
    auto socket_object = socket_info.Object;
    if (!socket_object) continue;

    // 최종 월드 행렬을 소켓 오브젝트에 적용
    XMFLOAT4X4 object_world_matrix = object->transform()->world_matrix(); // 플레이어 월드 행렬
    XMFLOAT4X4 socket_transform = mesh->get_socket_transform(socket_info.bone_name); // 뼈대 행렬

    XMFLOAT4X4 final_world_float4x4 = pair.second._localMatrix; // 검의 로컬 행렬

    // 검의 로컬 * 애니메이션에서 계산한 원하는 뼈대의 행렬 * 소켓을 들고있는 플레이어의 월드 행렬
    final_world_float4x4 = Matrix4x4::Multiply(final_world_float4x4, socket_transform);
    final_world_float4x4 = Matrix4x4::Multiply(final_world_float4x4, object_world_matrix);

    socket_info.Object->transform()->set_world_matrix(final_world_float4x4);
}
```

무기의 로컬 행렬 × 애니메이션 뼈대 행렬 × 캐릭터 월드 행렬의 계층적 연산 로직

```cpp
::std::shared_ptr<GameObject> add_connecting(const std::string& socket_name,
                                             const std::string& bone_name, const std::string& mesh,
                                             XMFLOAT3 local_pos = {0.f, 0.f, 0.f},
                                             XMFLOAT3 local_rotation = {0.f, 0.f, 0.f},
                                             XMFLOAT3 local_scale = {1.f, 1.f, 1.f});
```

소켓 연결 함수

```cpp
// 원하는 무기 붙이기
auto socket_component = hi_brute->get_component<SocketComponent>();
socket_component->add_connecting("ik_hand_l_sword", "hand_l", "Resource/Weapons/SM_Weapon_Sword__10/SM_Weapon_Sword__10.gltf",
    { 0.0623f, -0.8154f, 0.1643f }, { -10.f,90.f,-179.f }, { 2.f,2.f,2.f });
```

실제 사용 예시



(자체 제작 툴 화면) 소켓 컴포넌트를 이용하여 무기를 부착한 모습

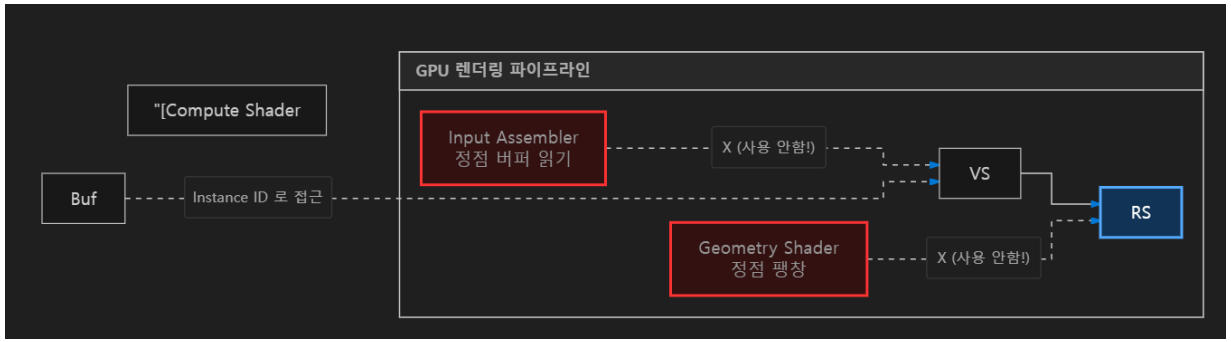


Compute Shader Particle System

[? 문제점] 수만 개의 파티클 이동 궤적을 CPU에서 계산하거나, 매 프레임 파티클 정점 버퍼(Vertex Buffer)를 생성해 GPU로 전송할 경우 심각한 병목 발생

[→ 해결책] Compute Shader 오프로드 및 Bufferless Procedural Particle 구현

- Compute Shader 오프로드(Offload): 수 만개의 파티클 위치 계산 로직을 CPU에서 완전히 분리하여, GPU의 수천 개 스레드(CS)로 100% 병렬 처리
- Bufferless 절차적 생성: 정점 버퍼를 파이프라인에 바인딩하지 않고, SV_VertexID를 활용해 Vertex Shader 내부에서 수학적으로 사각형(Quad) 정점을 실시간 생성
- 지오메트리 셰이더(GS) 병목 제거: Input Assembler(IA)의 메모리 참조 비용을 0으로 만들고, 병목이 심한 GS의 정점 팽창 오버헤드를 완벽히 우회하는 최신 렌더링 최적화 적용



정점 버퍼(IA)와 지오메트리 셰이더(GS)를 생략한 최적화된 파티클 렌더링 파이프라인

```hlsl
// 가상의 쿼드(사각형)를 위한 4개의 UV와 로컬 좌표
static const float2 QuadUVs[4] = { float2(0, 0), float2(1, 0), float2(0, 1), float2(1, 1) };
static const float2 QuadPos[4] = { float2(-0.5, 0.5), float2(0.5, 0.5), float2(-0.5, -0.5), float2(0.5, -0.5) };

VS_OUT VS_Particle(uint vI : SV_VertexID, uint instI : SV_InstanceID)
{
    VS_OUT Out;

    // 인스턴스 ID를 통해 컴퓨트 셰이더가 계산한 월드 위치를 가져옴
    float3 worldPos = g_ParticleBuffer[instI];

    // 카메라를 항상 정면으로 바라보도록 축(Right, Up) 계산 (Billboarding)
    float3 look = normalize(gvCameraPosition.xyz - worldPos);
    float3 right = normalize(cross(float3(0, 1, 0), look));
    float3 up = cross(look, right);

    // 0~3의 VertexID에 따라 사각형의 네 꼭짓점 위치로 벌려줌
    float current_size = g_Size;

    float2 qpos = QuadPos[vI] * current_size;
    worldPos += right * qpos.x + up * qpos.y;
```

VS 내부에서 정점을 생성하여 파티클을 제작하는 로직



파티클이 원하는 색상, 모양으로 모여드는 모습



한국게임학회 춘계 학술발표대회

논문 제목: 3D MO 액션 게임을 위한 glTF 2.0 표준 파서 및 스키닝 애니메이션 구현 최적화 연구

학술대회명: 2026년 한국게임학회 춘계 학술발표대회

연구 개요: 상용 엔진 및 범용 라이브러리(Assimp 등)에 의존하지 않고, DirectX 12 환경에 최적화된 독자적인 3D 에셋 파싱 시스템과 렌더링 파이프라인을 구축하여 메모리 효율과 연산 속도를 극대화한 연구

저자: 4인 (학생 3, 지도교수 1)

[핵심 연구 및 기여]

자체 glTF 파서: 포인터 오버헤드를 제거한 1차원 배열 구조로 CPU 캐시 히트율 극대화

메모리 최적화: 256Byte 정렬 기반 Linear Allocator를 직접 설계하여 메모리 단편화 원천 차단

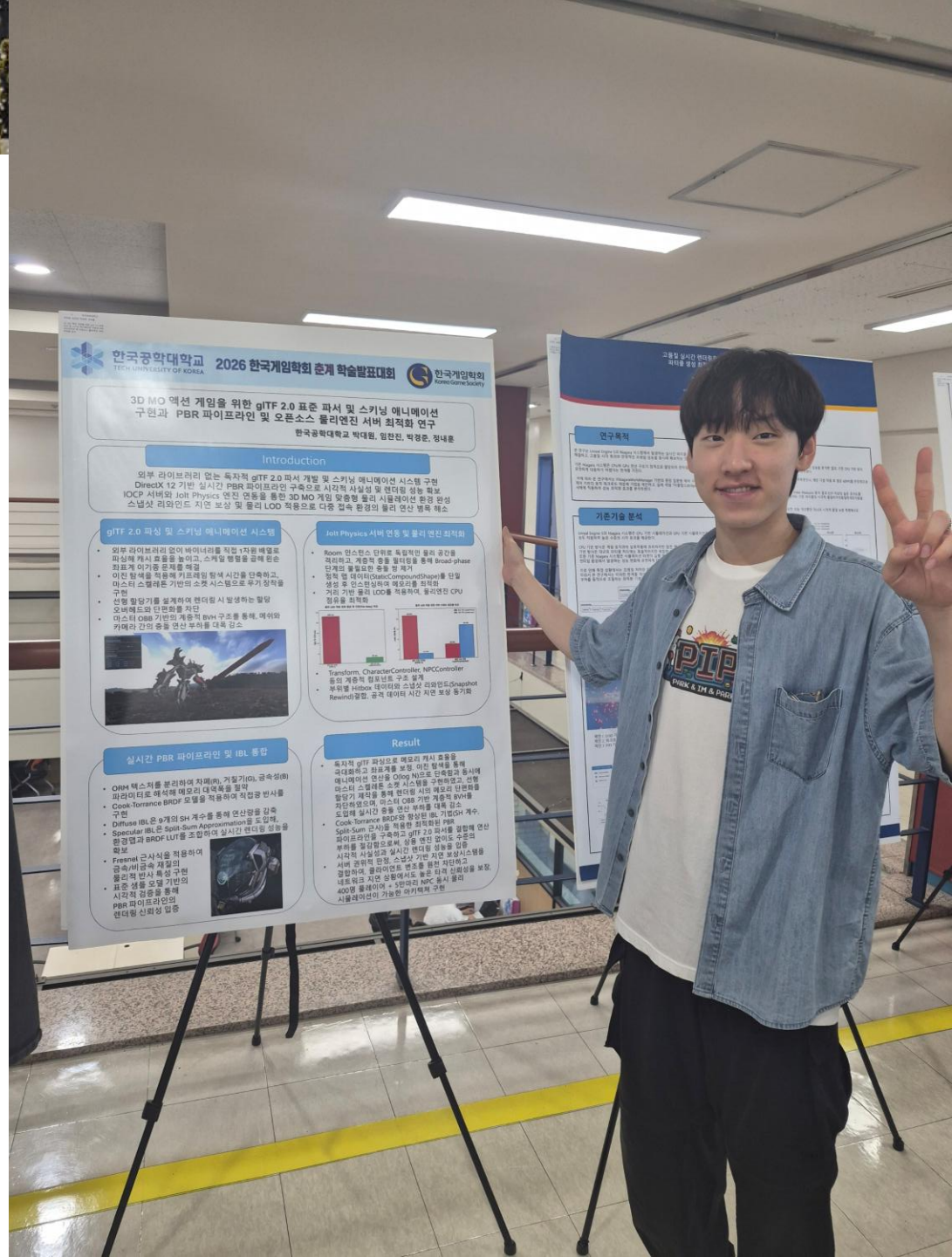
애니메이션 성능 개선: 이진 탐색을 도입하여 키프레임 탐색 비용을 기존 O(N)에서 O(log N)으로 단축

스켈레톤 및 소켓 시스템: 파츠 통합 렌더링을 위한 마스터 스켈레톤 기반 무기 부착 소켓 구현

충돌 검사 최적화: 마스터 OBB 기반의 BVH 2단계 검사 구조를 도입해 메쉬-카메라 간 충돌 연산 부하 감소

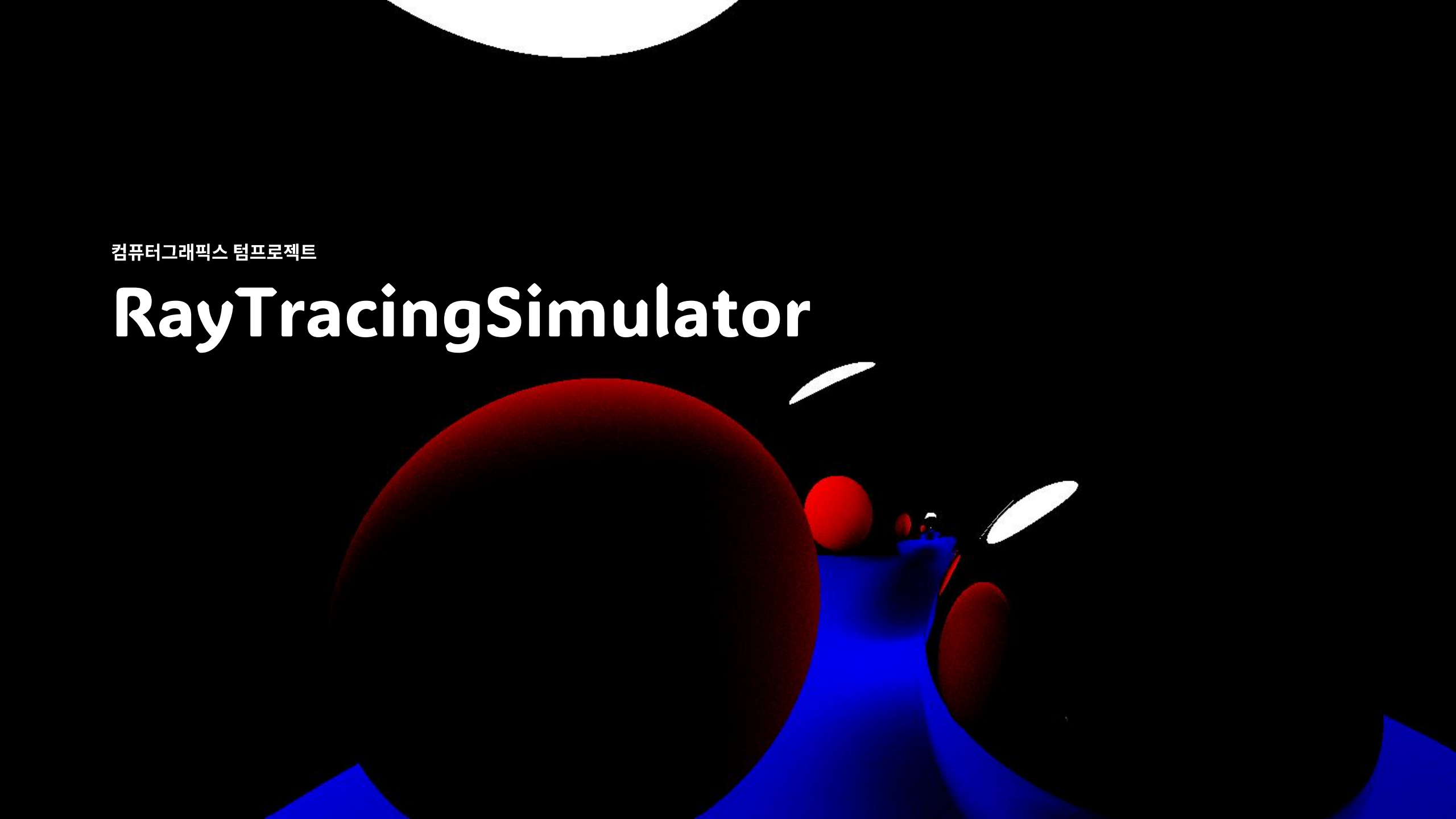
파티클 알고리즘: 무게중심 좌표계를 적용해 표면에 뭉침 없이 균일하게 생성되는 파티클 구현

논문PDF
https://drive.google.com/file/d/1W84MuxEyGpaSZkEP0vgDogjQdBBfCAUY/view?usp=sharing



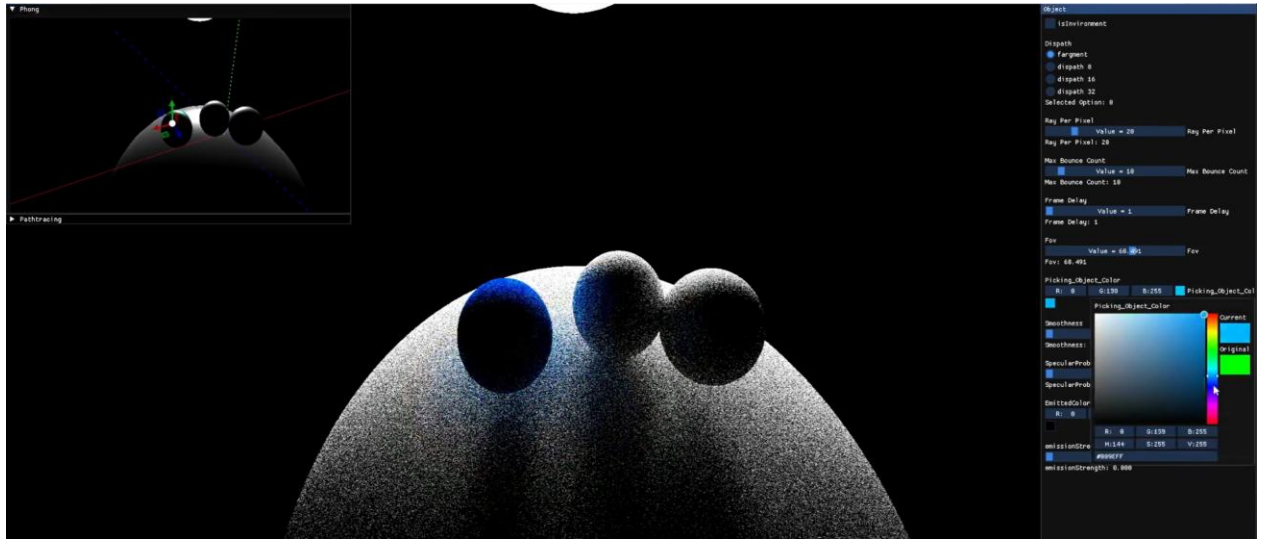
컴퓨터그래픽스 텀프로젝트

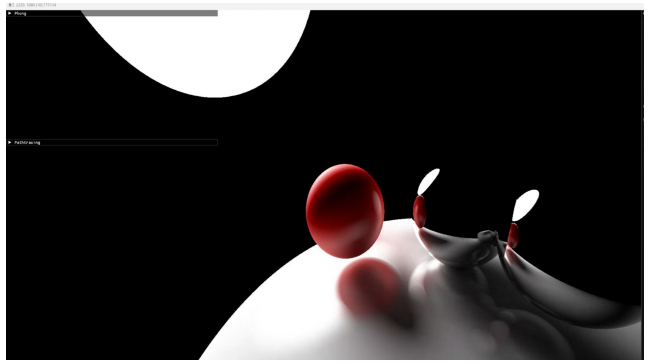
RayTracingSimulator

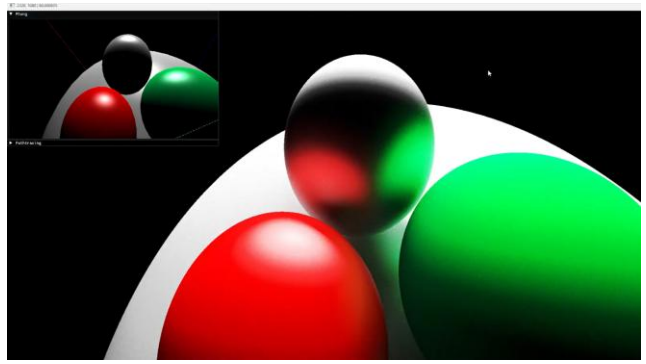


RayTracingSimulator(OpenGL)

프로젝트 개요



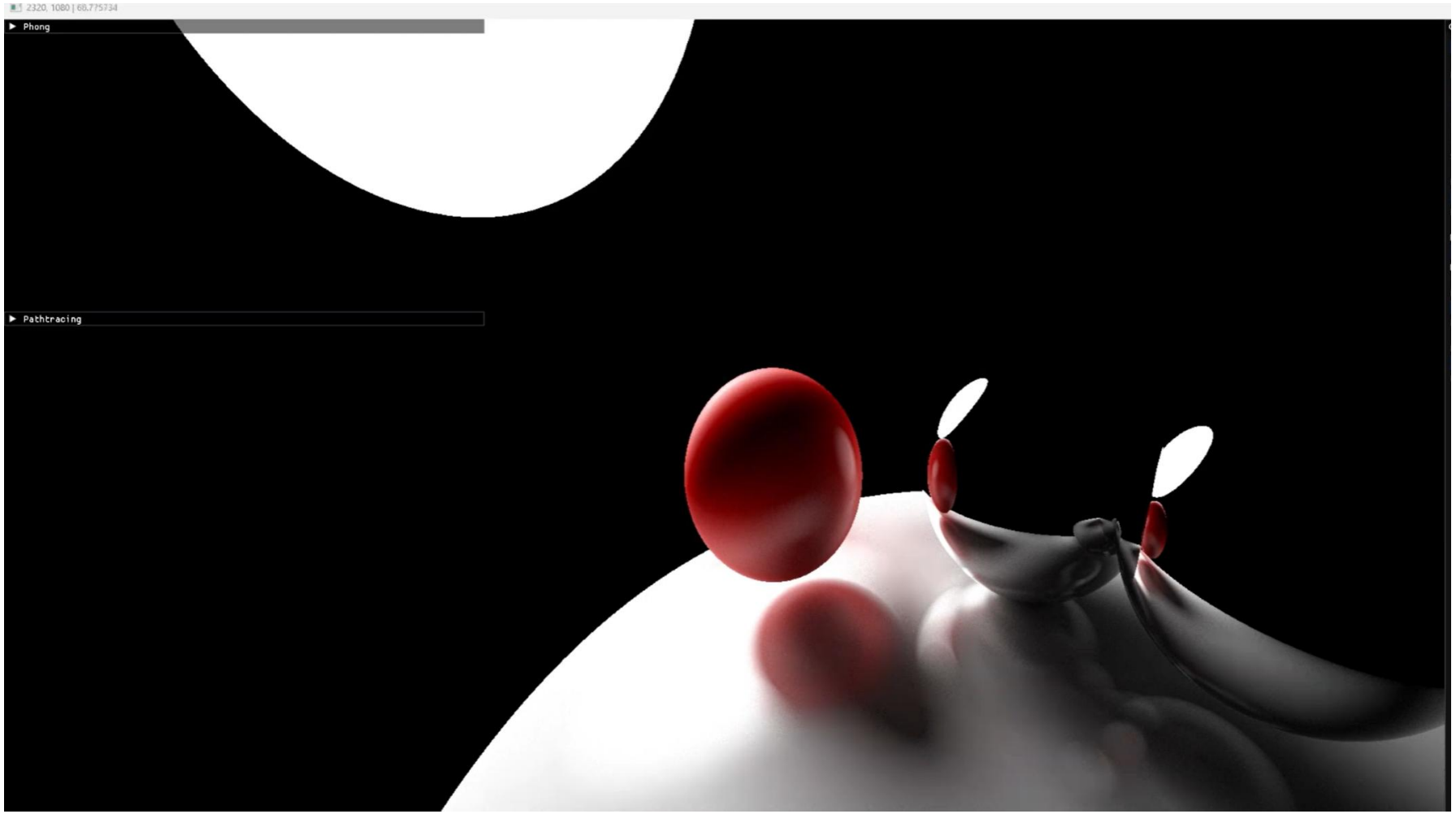




프로젝트 이름	Ray Tracing Simulator
장르	실시간 그래픽스 렌더링 시뮬레이터
설명	빛의 반사와 산란을 추적하는 실시간 레이트레이싱 시뮬레이터
개발 인원	2명 (클라이언트/그래픽스 프로그래밍 2)
담당 역할	실시간 렌더링 노이즈 최적화, 객체 속성 제어 툴 제작
개발 언어	C++20, GLSL (OpenGL Shading Language)
사용 툴 & 라이브러리	Visual Studio 2022, OpenGL, GLFW, GLM, ImGui (ImGuizmo), Git
활용 AI	Claude, Gemini (알고리즘 검증 및 디버깅 보조)
제작 기간	2024 / 12 ~ 2024 / 12 (약 2주)

RayTracingSimulator(OpenGL)

시뮬레이션 영상



https://youtu.be/zepNXOtQ1SM?si=GlrknuzDK_JXsJxZ

RayTracingSimulator(OpenGL)

실시간 렌더링 노이즈 최적화

[? 문제점] 실시간 레이트레이싱 특성상 픽셀당 광선을 한 번만 쏘면 노이즈가 매우 심하게 발생

[→ 해결책] 매 프레임 난수 시드를 변경하여 광선을 쏘고, 이전 프레임 버퍼와 가중치(1 / 누적 프레임) 연산을 통해 색상을 누적하는 누적 프레임 버퍼 방식을 통해 노이즈 제거

[? 문제점] 화면이 정지해 있을 때는 노이즈가 사라지지만, 카메라가 이동하면 이전 프레임의 잔상이 남는 고스팅(Ghosting) 현상이 발생

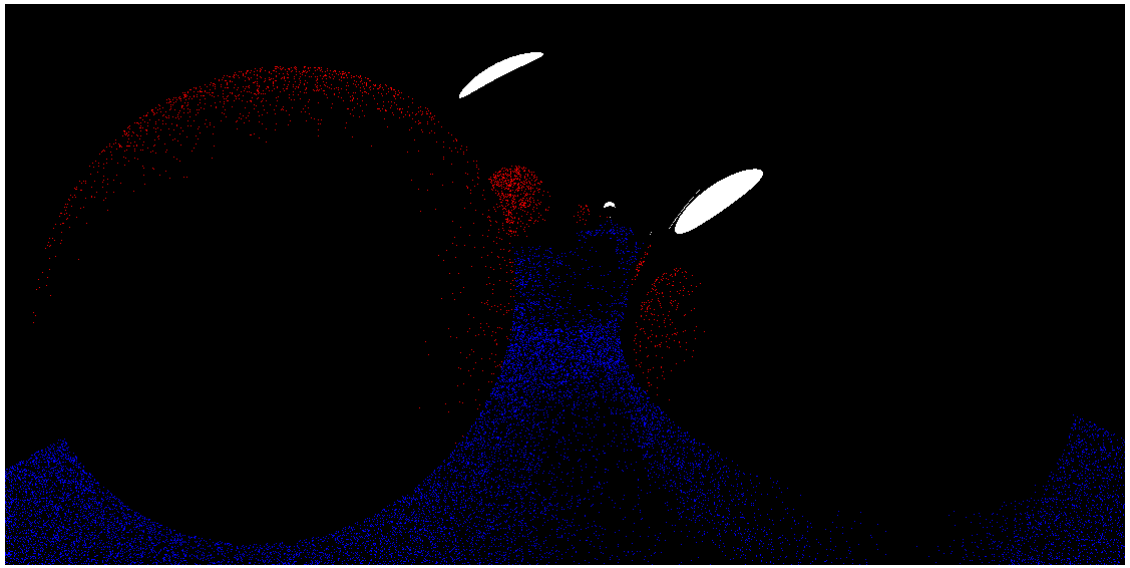
[→ 해결책] 카메라의 View/Projection 행렬이 변경될 때마다 누적 프레임 카운트를 0으로 초기화하여 잔상 문제 해결

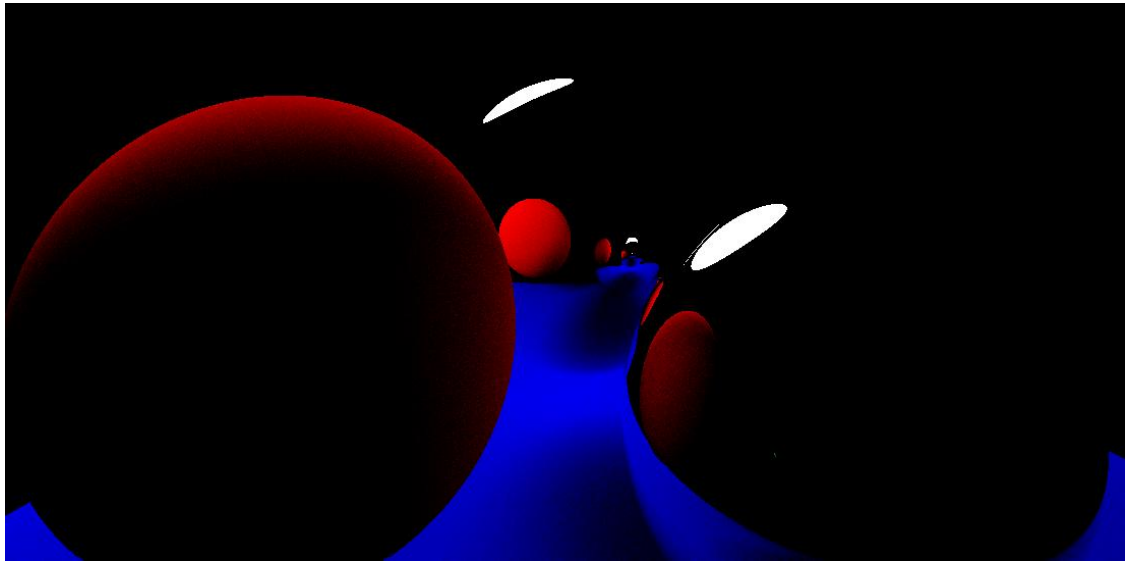
```
uint rngState = pixelIndex + numRenderedFrame * 925904;
```

매 프레임 다른 난수를 발생시켜 레이트레이싱 궤적을 분산

```
FragColor = mix(oldTexColor, newTexColor, 1 / float(numRenderedFrame + 1));
```

1/N 가중치로 과거 프레임과 현재 프레임의 색상을 섞어 노이즈를 수렴





노이즈가 줄어드는 모습

RayTracingSimulator(DirectX 12, DLSS RR)

실시간 패스 트레이싱 고도화 및 신경망 광선 재구성 (DLSS RR)

[? 문제점] 시간적 누적 방식은 카메라 이동 시 고스팅 방지를 위해 프레임 카운트를 0으로 리셋하므로, 카메라 이동 중에는 화면이 심한 1 SPP 노이즈로 되돌아가는 한계 발생

[→ 해결책] DirectX 12 Compute Shader 기반 Full GI 파이프라인 구축 및 NVIDIA DLSS 3.5 Ray Reconstruction(신경망 추론) 연동을 통한 실시간 노이즈 제거

```
struct Material {
    float4 color;      // 기본 색상 (Base Color RGB)
    float4 emission;   // 발광 색상(RGB) 및 발광 강도(W Intensity)
    float4 parameters; // x: 거칠기(Roughness),
                       // y: 금속성(Metallic),
                       // z: 굴절률(IOR),
                       // w: 투명도(Transmission)
};
```

발광, 굴절 등을 고려한 재질

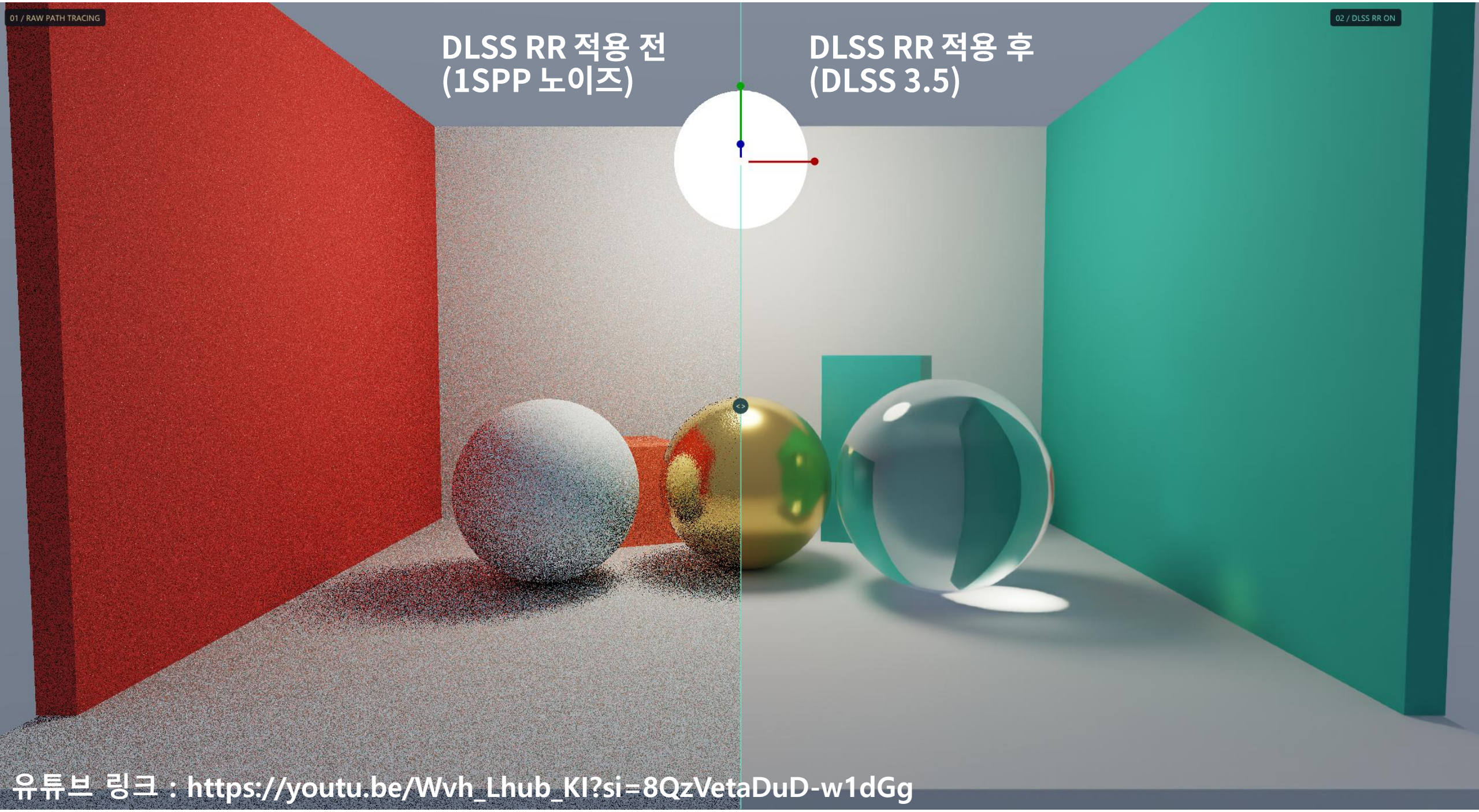
- DirectX 12 Full GI 고도화: OpenGL 파이프라인을 DX12 Compute Shader로 재구축하고 GGX PBR, 유리 굴절 고려, NEE 광원 샘플링 적용
- 1-Pass 7대 가이드 추출: 컴퓨트 셰이더 1회 패스에서 DLSS RR에 필요한 가이드 버퍼들 동시 추출
- DLSS 3.5 텐서 코어 추론: 1 SPP 노이즈와 가이드 버퍼를 텐서 코어에 전달하여, 이동 중에도 단 1프레임 만에 노이즈 없는 화면 실시간 재구성

```
// DLSS 3.5 Ray Reconstruction 입력 7대 물리 가이드 버퍼
RWTexture2D<float4> Output              : register(u0); // Full GI 누적 프레임 버퍼
RWTexture2D<float4> NoisyColor          : register(u1); // 1 SPP 노이즈 원본
RWTexture2D<float4> DiffuseAlbedo       : register(u2); // 물체 기본 색상
RWTexture2D<float4> SpecularAlbedo      : register(u3); // 표면 반사 색상
RWTexture2D<float4> NormalRoughness     : register(u4); // 표면 법선 벡터 + 거칠기
RWTexture2D<float>  ViewDepth           : register(u5); // 카메라 선형 깊이
RWTexture2D<float2> MotionVectors       : register(u6); // 카메라 이동 모션 벡터
RWTexture2D<float>  SpecularHitDistance : register(u7); // 2차 반사 광선 도달 거리
```

DLSS RR을 위한 가이드 버퍼들



파이프라인 구성


01 / RAW PATH TRACING
DLSS RR 적용 전
(1SPP 노이즈)
DLSS RR 적용 후
(DLSS 3.5)
02 / DLSS RR ON
유튜브 링크 : https://youtu.be/Wvh_Lhub_KI?si=8QzVetaDuD-w1dGg

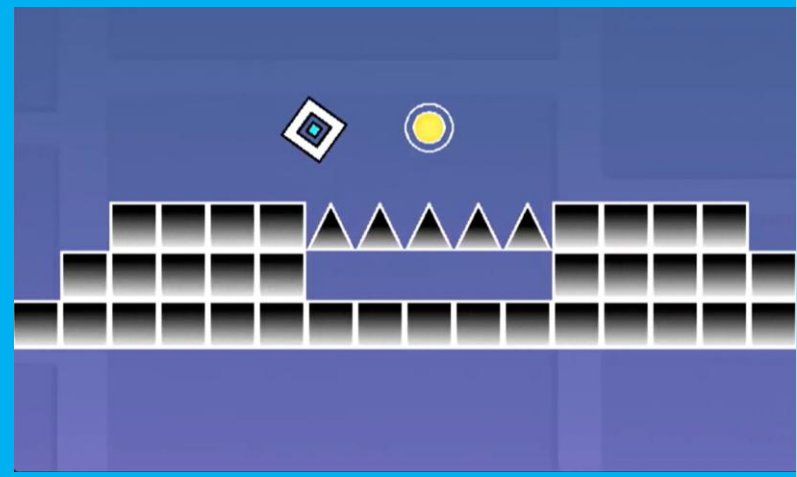
기타 프로젝트

1. Riffle Effect 모작 (2.5D, DirectX 9)
2. 2D MMORPG IOCP Server (2D, WinAPI, IOCP)
3. Moana Tower Defense (3D, 언리얼)
4. Sonic the Hedgehog 모작 (2D, WinAPI)
5. CupHeadBanging (2D, Python)
6. Geometry Dash 모작 (2D, WinAPI)
7. TheGreatestNature (3D, 언리얼)





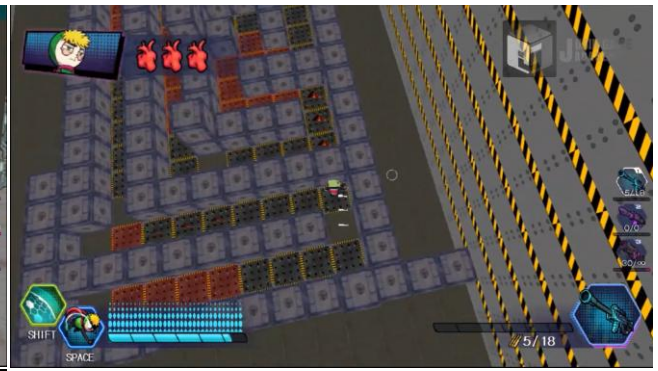
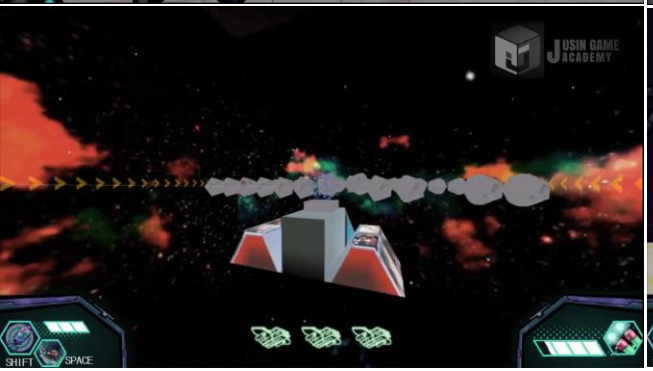
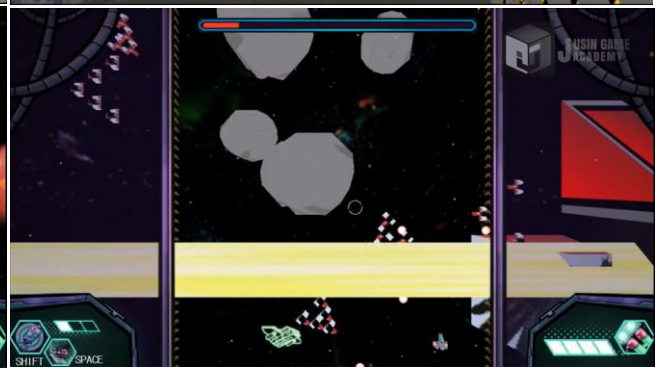
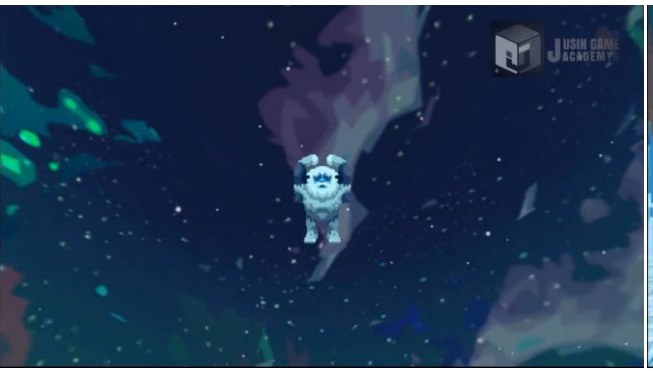








Riffle Effect


프로젝트 이름	Riffle Effect
장르	2.5D 탑뷰 & TPS 시점 전환 슈팅 액션 게임
설명	인디 게임 '리플 이펙트(Riffle Effect)' 를 재구성하여, 새로운 기믹 및 보스 몬스터를 추가한 액션 슈팅 게임
개발 인원	4명 (클라이언트 프로그래밍 4명)
담당 역할	필드/보스 몬스터 패턴 및 기믹 구현, 탑뷰 및 TPS 카메라 시점 전환 시스템 구현, 우주선 레벨 슈팅 전투 로직 및 엔딩 인게임 연출 제작
개발 언어	C++20, HLSL
사용 툴 & 라이브러리	Visual Studio 2022, DirectX 9, ImGui (ImGuizmo), WinMerge
제작 기간	2024 / 01 ~ 2024 / 02

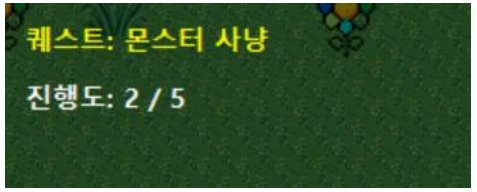
https://youtu.be/cFeB0f-qD6Y?si=5i_OeCtOlstySU4b

2D MMORPG IOCP Server

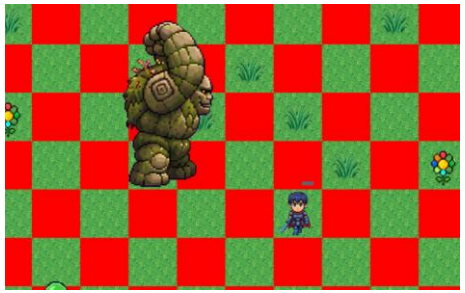


인게임 화면

(채팅, 멀티 유저, 필드 몬스터, UI)



퀘스트



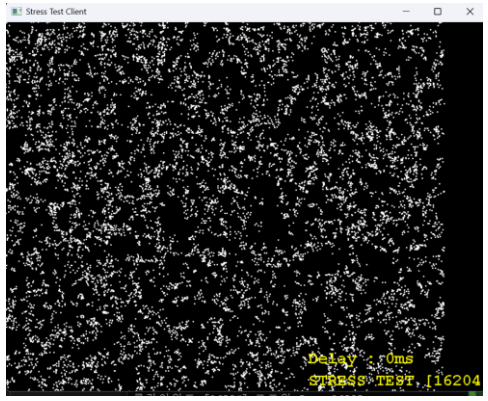
보스 몬스터

(Lua 스크립트를 사용)

DAETOP₩SQL...bo.Players

	name	experience	player level	quest id	quest progress
▶	asdf	245	3	NULL	NULL
	asfsfd	200	3	NULL	NULL
	dfsf	45	1	NULL	NULL
	Good	150	4	NULL	NULL
	PakrDaeWon	350	7	NULL	NULL
	ParkDaeWom	0	1	0	0
	ParkDaeWon	1859	60	0	0
	PDW	9	5	0	0
*	NULL	NULL	NULL	NULL	NULL

데이터 베이스 구성



서버 딜레이, 동접자 수

플레이어들의 실시간 위치를

표시한 클라이언트

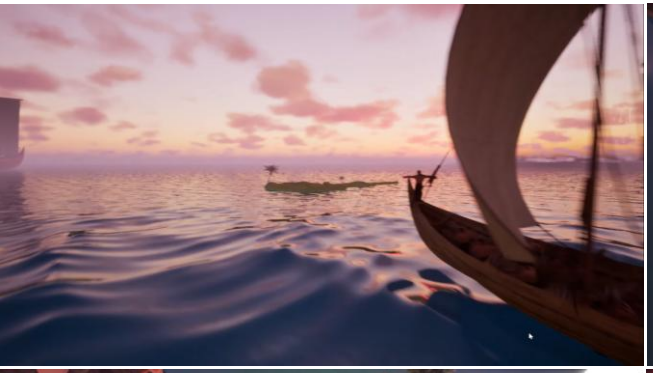
프로젝트 이름	2D MMORPG IOCP Game Server
장르	2D MMORPG
설명	WinAPI 기반 2D 게임 클라이언트와 Windows IOCP 기반 고성능 서버를 직접 구축하고, 5,000명 동접 부하 테스트 및 20만 몬스터 시야를 최적화한 프로젝트
개발 인원	1명 (1인 개발)
담당 역할	WinAPI 2D 클라이언트 제작: 카메라 워킹, 씬·오브젝트 매니저 및 서버 패킷 처리 IOCP 서버 및 시야 최적화: 10,000개 섹터 AOI 관리 및 5,000명 동접 부하 검증 Lua 스크립팅 및 DB 연동: 20만 몬스터·보스 AI 구현 및 MS SQL 유저 데이터 저장
개발 언어	C++20, Lua, SQL
사용 툴 & 라이브러리	Visual Studio 2022, MS SQL Server, Winsock2 (IOCP), ODBC, Lua
활용 AI	Claude, Gemini, Antigravity (알고리즘 검증 및 디버깅 보조)
제작 기간	2026 / 05 ~ 2026 / 06

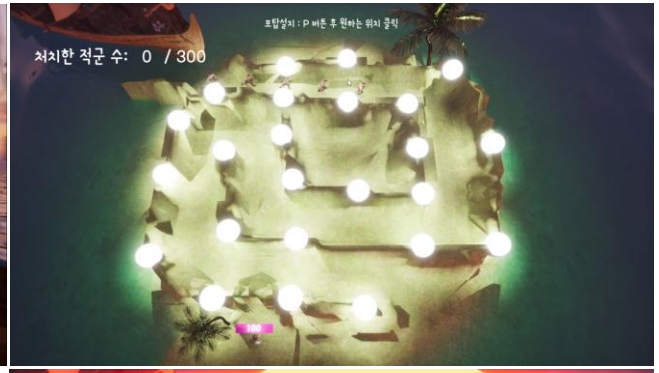
텀 프로젝트 발표자료 : https://drive.google.com/file/d/14xXgj7LztM5sHM35zbJ0P9fKiMcPouYw/view?usp=sharing

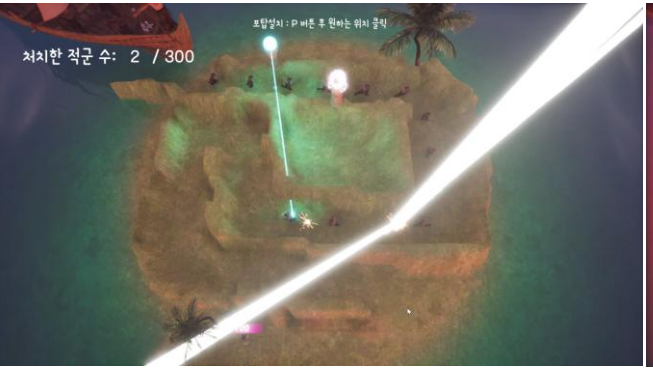
Moana Tower Defense













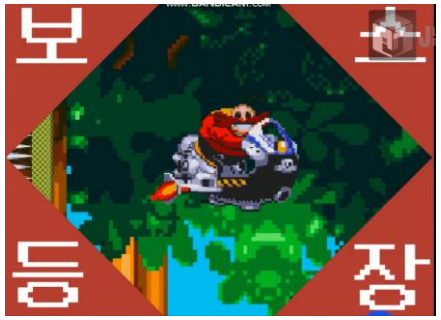
프로젝트 이름	Moana Tower Defense
장르	타워 디펜스
설명	영화 '모아나'를 모티브로 하여, 정적인 타워 디펜스의 틀을 깨고 역동적인 카메라 워킹과 타워의 타격감을 더한 디펜스 게임
개발 인원	1명 (1인 개발)
담당 역할	AI 및 전투 로직: NavMesh 기반 적군 경로 탐색 알고리즘 구현 및 3종의 고유 타워 시스템 설계 게임 연출 : 워터 플러그인, 랜드스케이프, 카메라 레일을 활용한 컷씬 제작, 각 타워의 타격감(시각 효과) 극대화
개발 언어	Blueprint
사용 툴 & 라이브러리	Unreal Engine 5, Mixamo
활용 AI	MeshyAI (모아나 NPC 모델 제작)
제작 기간	2025 / 06 ~ 2025 / 07

https://youtu.be/EAaFWKJy8-c?si=-9c4y0dLxe3j963L

Sonic the Hedgehog







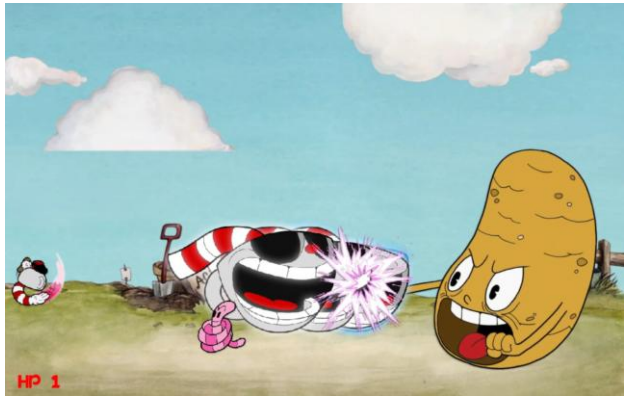




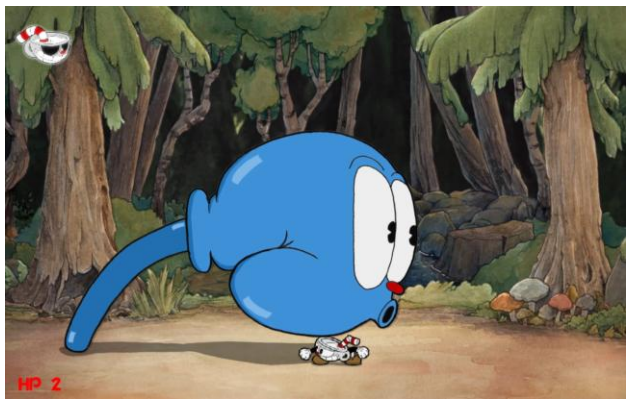

프로젝트 이름	Sonic the Hedgehog
장르	2D 횡스크롤 플랫포머 액션 게임
설명	소닉 특유의 속도감 있는 물리 이동과 화려한 연출을 WinAPI 기반의 2D 환경에서 심도 있게 재구성한 액션 게임
개발 인원	1명 (1인 개발)
담당 역할	자체 맵 에디터(Tool) 개발: 맵 툴을 직접 제작하여 모든 맵의 레벨 디자인을 독자적으로 설계 및 구축 게임 플레이 및 연출 (Gameplay): 고속 이동 물리 로직, 보스전 패턴, 플레이어 상태 변신, 추격전, 동료 모드 등 다양한 인게임 시스템과 시각적 연출 구현
개발 언어	C++20
사용 툴 & 라이브러리	Visual Studio 2022, WinAPI, FMOD
제작 기간	2023 / 10 ~ 2023 / 11

https://youtu.be/j4mQUpVuXnc?si=sDESffBFRREsv306

CupHeadBanging











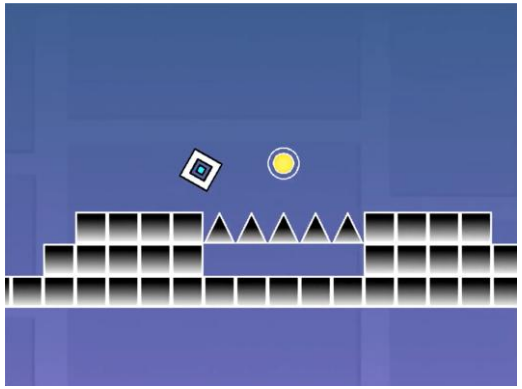


https://youtu.be/lu98Zv6jaDE?si=X_epXKmIyldpYftK

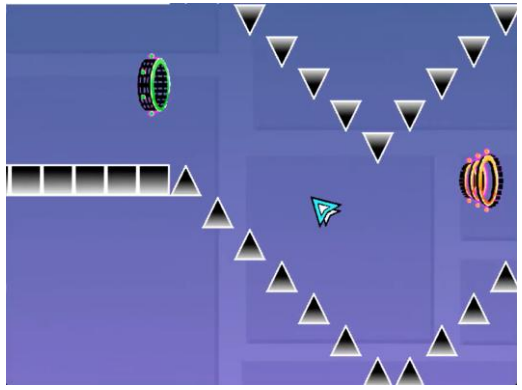
프로젝트 이름	CupHeadBanging
장르	2D 런앤건(Run and Gun) 보스 러시 액션 게임
설명	고전 애니메이션 스타일의 하드코어 액션 게임 '컵헤드(Cuphead)' 를 재구성하여, 정교한 충돌 처리와 다단계 보스전 로직을 파이썬 환경에서 제작한 프로젝트
개발 인원	1명 (1인 개발)
담당 역할	프레임워크 설계: Manager 패턴(Object, Stage 등)을 도입한 게임 루프 및 씬 제어 구조 구축 보스 AI 구현: 페이즈별로 공격 패턴이 변화하는 2종의 보스 몬스터 로직 설계 충돌 및 애니메이션: 자체 매니저를 통한 정밀한 히트박스 연산과 프레임 단위 애니메이션 처리
개발 언어	Python
사용 툴 & 라이브러리	PyCharm, Pico2D
제작 기간	2024 / 09 ~ 2024 / 12

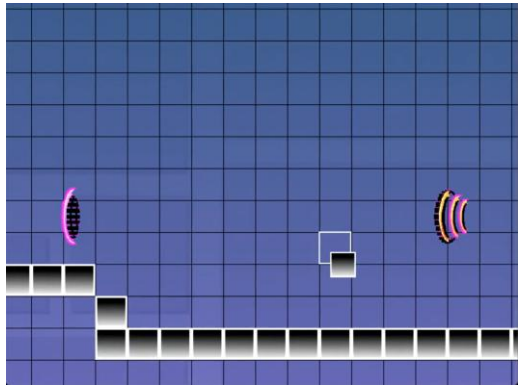
Geometry Dash

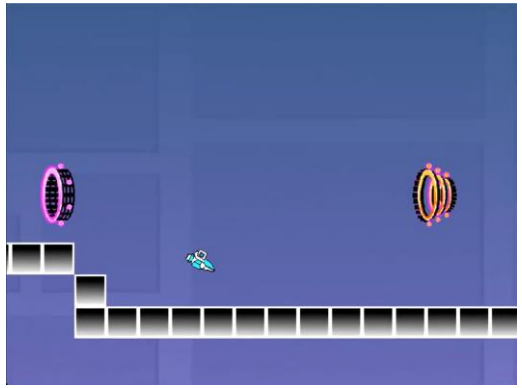








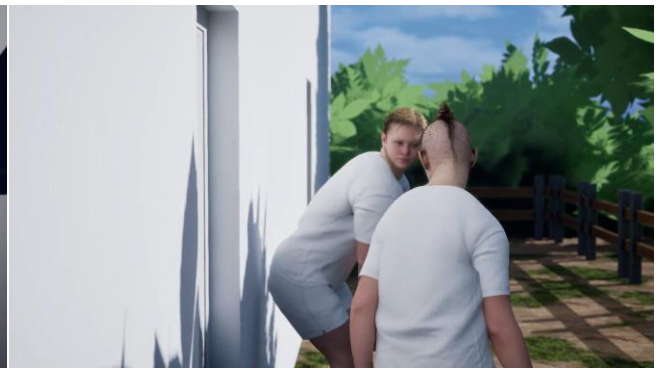
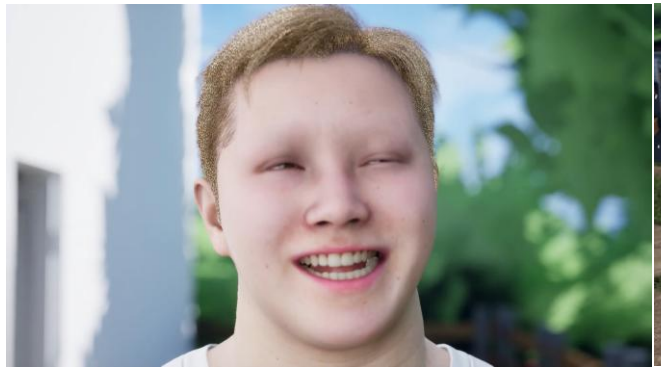


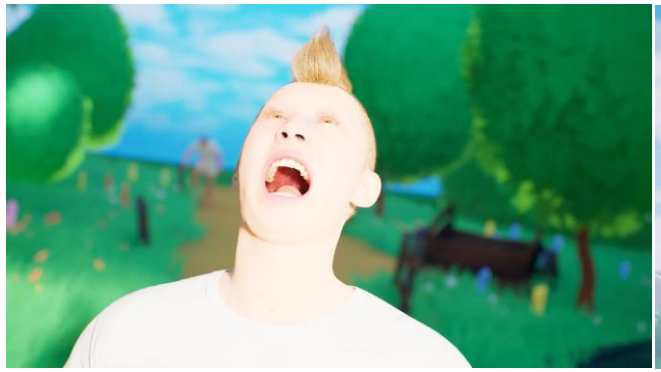


프로젝트 이름	Geometry Dash
장르	2D 리듬 플랫포머 게임
설명	유저가 직접 즐거움을 창조할 수 있도록, 플레이 시스템과 커스텀 맵 에디터를 완벽히 통합하여 개발한 리듬 플랫포머 게임
개발 인원	2명 (클라이언트 프로그래밍 2명)
담당 역할	자체 프레임워크 설계: 게임 루프, 씬/리소스 관리 등 2D 코어 프레임워크 전담 구축 인게임 맵 에디터: 실시간 맵 커스터마이징 및 즉시 플레이 테스트 시스템 구현 게임 플레이 및 기믹: 물리 연산 및 특수 모드(비행선, 지그재그) 전환 로직 구현
개발 언어	C++20
사용 툴 & 라이브러리	Visual Studio 2022, WinAPI, FMOD
제작 기간	2024 / 05 ~ 2024 / 06

https://youtu.be/fucmZyc_uyM?si=LUe5tRA9mOWZo3zY

TheGreatestNature




프로젝트 이름	TheGreatestNature
장르	3D 시네마틱 영상 연출
설명	언리얼 엔진 5의 메타휴먼(MetaHuman)과 시퀀서를 적극 활용하여, 고품질 3D 애니메이션 컷씬과 카메라 연출 기법을 연구한 프로젝트
개발 인원	2명 (공동 개발)
담당 역할	메타휴먼 및 페이셜 캡처 연동: 얼굴의 특징을 살린 고품질 MetaHuman 아바타 제작 및 스마트폰 Live Link Face 앱을 연동한 실시간 페이셜 캡처 적용 시네마틱 컷씬 연출: 레벨 시퀀서를 활용한 역동적인 카메라 워킹, 씬 전환, 생동감 있는 애니메이션 적용
개발 언어	Blueprint
사용 툴 & 라이브러리	Unreal Engine 5, MetaHuman, Live Link Face
제작 기간	2025 / 11 ~ 2025 / 12

https://youtu.be/9aAs1NV_c7Q?si=OHBe_pAX1ypEgC2Z

Thank you!

함께 성장하는 프로그래머가 되겠습니다.
감사합니다!
박대원

